

ECON1059 Seminar 5

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Tuples

- A **tuple** is an **ordered** and **immutable** collection of elements.
- Created using **parentheses** `()`, and can store **different data types** (e.g., `("world", 2)`).
- For a **single element**, write `(2,)` with a trailing comma — `(2)` is just an integer, not a tuple.

Why Use Tuples?

- **Prevent unintended modification:** Use a tuple when the data should not change.
- **Multiple return values:** Functions that return more than one value automatically return them as a **tuple**, which can then be unpacked (e.g., `x, y = func()`).

```
In [1]: # Example 1: Returning multiple values as a tuple
def function_name(param1, param2):
    add = param1 + param2
    multiply = param1 * param2
    return add, multiply # returns a tuple (add, multiply)

results = function_name(1, 2)
print(results)
```

`(3, 2)`

```
In [2]: # Tuples are immutable
results[0] = 8 # TypeError
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[2], line 2
      1 # Tuples are immutable
----> 2 results[0] = 8

TypeError: 'tuple' object does not support item assignment
```

```
In [3]: # Example 2: Unpacking (assign the value)
sum_result, product_result = function_name(3, 4)

print(sum_result)      # Output: 7
print(product_result)  # Output: 12
```

7
12

- Create any tuple that you like, access one of its values.
- Can a tuple contain more than one datatype?
- If a tuple contains one or more lists, can you change a value in such a list. Can you replace any lists?

```
In [4]: t = (1, "b") # Yes, it's possible to contain more than one datatype
```

```
In [5]: # Yes, values within mutable objects within a tuple can be changed.
# Reason, we are not "changing" the elements within the tuple, a list,
# But elements within those objects.
t = ([1,2], [3,4])
t[0][1] = 55
print(t)
```

([1, 55], [3, 4])

```
In [6]: # But you cannot replace the highest level item in the tuple itself
t[1] = [44, 55]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 2
      1 # But you cannot replace the highest level item in the tuple itself
----> 2 t[1] = [44, 55]
```

```
TypeError: 'tuple' object does not support item assignment
```

zip()

- `zip()` is a built-in function that **combines two or more sequences** (e.g., lists, tuples, **strings**, range) into **tuples**.
- Elements are paired **by their positions** (1st with 1st, 2nd with 2nd, etc.).
- It behaves like a **one-time parcel**: once unpacked (e.g., converted to a list or looped over), it becomes empty and must be recreated to use again.

```
In [7]: ## Example 3: Basic usage of zip()
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]

combined = zip(names, scores)

print(combined)          # A lazy zip object (iterator) – like a parcel: you must open it to see what's inside
print(type(combined))    # Notice this is neither a list nor a tuple
print(list(combined))    # Convert to a list → opens the parcel and consumes the iterator

# Possible Output:
# <zip object at 0x000001A3B8F...>
```

```
# <class 'zip'>
# [('Alice', 85), ('Bob', 92), ('Charlie', 78)]
```

```
<zip object at 0x104954600>
```

```
<class 'zip'>
```

```
[('Alice', 85), ('Bob', 92), ('Charlie', 78)]
```

```
In [8]: print(list(combined))      # Empty – the parcel has been opened; re-zip to get items again
```

```
[]
```

```
In [9]: ## Example 4: zip() stops at the shortest sequence
names = ["Alice", "Bob", "Charlie"]
scores = [90, 85]

for name, score in zip(names, scores): # Unpacking
    print(name, score)
# Output:
# Alice 90
# Bob 85
```

```
Alice 90
```

```
Bob 85
```

```
In [10]: ## Example 5: Using strict=True
names = ["Alice", "Bob", "Charlie"]
scores = [90, 85]

for name, score in zip(names, scores, strict=True):
    print(name, score)

# Output:
# ValueError: zip() argument 2 is longer than argument 1
```

```
Alice 90
```

```
Bob 85
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 5
      2 names = ["Alice", "Bob", "Charlie"]
      3 scores = [90, 85]
----> 5 for name, score in zip(names, scores, strict=True):
      6     print(name, score)

ValueError: zip() argument 2 is shorter than argument 1
```

```
In [11]: ## Example 6: It also works for strings and range! String is a sequence of characters after all.
word = "cat"
for ch in word:
    print(ch)
for ch, pos in zip(word, range(3)):
    print(ch, pos)
```

```
c
a
t
c 0
a 1
t 2
```

- Use the `zip()` function to create a list of tuples of the form (1,10), (2, 11), (3, 12), (4, 13).

```
In [12]: # Basic Solution
new_list = []

for item in zip(range(1, 5), range(10, 14)):
    print(item)
    new_list.append(item)

print(new_list)
```

```
(1, 10)
(2, 11)
(3, 12)
(4, 13)
[(1, 10), (2, 11), (3, 12), (4, 13)]
```

- Transform `zip()` object using `list()`

```
In [13]: iterable = zip(range(1, 5), range(10, 14))
print(type(iterable)) # Notice this is neither a list nor a tuple
li = list(iterable)
print(li)
```

```
<class 'zip'>
[(1, 10), (2, 11), (3, 12), (4, 13)]
```

Comment.

- To get a list of lists (instead of a list of tuples), you can adapt the "basic solution" from above that relies on a for loop
- The `list(iterable)` approach from the second solution cannot be used to create a list of lists, as it only converts the most outside object, here the iterable, to a list, but not the tuples that make up this iterable.
- We'll learn a shortcut later on in the course to create lists of lists.

enumerate()

- `enumerate()` is a built-in function that adds an automatic **index counter** to an iterable.
- It returns **(index, element)** pairs, allowing you to access both the position and the value in a loop.

```
In [14]: ## Example 6

fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

```
0 apple
1 banana
2 cherry
```

Docstring

A **docstring** (documentation string) is a string literal used to describe a Python module, function, class, or method. It is written inside **triple quotes** `"""` and placed **immediately after the definition**.

Writing Docstrings

- Enclosed in triple quotes, e.g., `"""My docstring"""` (single-line example).
- Multi-line docstrings should end with `"""` on a **new line**.
- Followed by a **blank line**.
- Should not repeat the name of the function/class — instead, explain **what it does** and **what it returns**.

Google Style Guide

- Recommends adding a docstring to any function, method, etc., that meets at least one of the following:
 - Part of the public API
 - Non-trivial in size
 - Contains non-obvious logic
- A well-structured docstring typically includes:
 - General description — What the function does
 - Arguments — The parameters it takes
 - Returns — What it outputs
 - Raises (exceptions) — Possible errors

```
In [15]: ## Example 7 Google Docstring Style
def add_numbers(a, b):
    """Add two numbers together.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: The sum of a and b.

    Raises:
        TypeError: If either input is not numeric.
    """
    return a + b
```

```
In [16]: # Get the docstring
help(add_numbers)
```

Help on function add_numbers in module __main__:

```
add_numbers(a, b)
  Add two numbers together.

  Args:
    a (int or float): First number.
    b (int or float): Second number.

  Returns:
    int or float: The sum of a and b.

  Raises:
    TypeError: If either input is not numeric.
```

```
In [17]: # Get the docstring
print(add_numbers.__doc__)
```

Add two numbers together.

```
  Args:
    a (int or float): First number.
    b (int or float): Second number.

  Returns:
    int or float: The sum of a and b.

  Raises:
    TypeError: If either input is not numeric.
```

- Write an example where you find the index for the first instance of some value in a list.
- Next, turn this into a general function. Your function should include a docstring. Write both a very short docstring as well as one that follows the Google Styleguide
- Note: if you check the build-in list-functions, you will see this exists already. But let's write our own function anyways as practise

```
In [18]: def find_first_instance_in_list(li, value_to_find):
        """Returns the index of the first occurrence of a value in a list."""

        for i, val in enumerate(li):
            if val == value_to_find:
                return i

        return None
```

```
In [19]: # And with the long - and indeed much more helpful - docstring:
```

```
def find_first_instance_in_list(li, value_to_find):
    """Finds the index of the first occurrence of the value in a list.

    Args:
        li: The list to be searched.
        value_to_find: The value to locate within the list.

    Returns:
        The index of the first occurrence of the value, or None if the
        value is not found.
```

```

"""

for i, val in enumerate(li):
    if val == value_to_find:
        return i

return None

```

Testing our function:

```

In [20]: li = ["a", 5, 9, "abc"]
print(f"Index for entry = 9 is: {find_first_instance_in_list(li, 9)}")

```

Index for entry = 9 is: 2

```

In [21]: print(f"Entry = 100 returns: {find_first_instance_in_list(li, 100)}")

```

Entry = 100 returns: None

- One interesting question is how we should deal with the case when the function doesn't find an instance of the value it's searching for
- In the above, I chose `None` over returning `False`, `len(li)` or `-1`. I leave it to you to explore why the alternative three options aren't ideal. (Hint, depending on the problem, you may want to use the index on the list again; moreover, `False` is the same as `0`).
- `False` → Bad: same as `0` → could mean index 0.
- `len(li)` → Misleading: it's the list length, not "not found".
- `-1` → Bad: valid index → refers to last element.

Built-in `.index()` function.

- The inbuilt function handles this differently. The documentation (<https://docs.python.org/3/tutorial/datastructures.html>) says:

"`list.index(x[, start[, end]])` Return zero-based index in the list of the first item whose value is equal to `x`. Raises a `ValueError` if there is no such item." (discussion regarding optional `[end]` argument skipped)"

- To understand this, let's try it out:

```

In [22]: li = [5, 6, 7]
print(li.index(7))

```

2

```

In [23]: print(li.index(8))

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 print(li.index(8))

ValueError: 8 is not in list

```

- The upside of this implementation: you won't incorrectly use some index again that cannot get what you want it to get.
- The downside: you need to build your code around this possibility, otherwise it will break.

Hangman Game

- After last week's "Guess the Number" Game, we will try to code the much more complicated Hangman Game in the Problem Set
- Please read for detail: [https://en.wikipedia.org/wiki/Hangman_\(game\)](https://en.wikipedia.org/wiki/Hangman_(game))
- I'll keep instructions minimal. That said, you may want to skip drawing people and use simple counters instead.
- This game requires a lot of moving parts.
 - It will be very difficult (for some of you)
 - Sketch out what you need; then sketch out how to solve each of those
 - Start small! Code individual items and test them separately. Generalize as you go along
 - Try to work on this for up to 2 hours :)
- Advanced: for those of you who found this somewhat manageable, go back to your code after it works and write it cleaner.

IF YOU ARE READING THE SLIDES-VERSION OF THE SOLUTIONS, PLEASE CONSULT OTHER OUTPUT FORMATS FOR THE HANGMAN GAME CODE

General Algorithm of Hangman

- [1] A secret word is selected, and numbers of letters are indicated as follows:
 - " _ _ _ _ _ "
- [2] Guessing Starts, which has the following structure:
- [3] If 7 misses: GameOver, otherwise
 - [3.1] guess a letter
 - [3.2] check if letter is part of secret word;
 - if true, write letter at the correct place of " _ _ _ _ _ "
 - if false, increase misses by 1;
 - advanced features: track wrong letters
 - advanced features: draw next part of the gallows
 - go back to [3].

```
In [ ]: secret_word = "hello" # Set word to test out our functions

# Create a list of the same length as our secret word, that contains "False" for each element
# as initially, no letter has been guessed correctly.
guessed_correctly = len(secret_word)*[False]
past_guesses = [] # To track guesses
total_misses_allowed = 6 # Do not change
total_misses = 0 # Tracks incorrect answers

while total_misses < total_misses_allowed:
    # A. Print secret word
    to_print = ""
    for letter, guess in zip(secret_word, guessed_correctly):
        if guess == True:
            to_print += letter + " "
        else:
            to_print += "_ "
    print(to_print)
```



```

# B. Print guesses already made; simplest possible implementation
print(f"Already guessed: {past_guesses}.\n")

# C. Elicit new guess
guess = input(f"Please make a guess ({total_misses_allowed - total_misses} mistakes left):\n")

# D. Check guess
guess_is_correct = False

for i, secret_word_char in enumerate(secret_word):
    if guess == secret_word_char:
        guess_is_correct = True
        guessed_correctly[i] = True
        # Not breaking the loop since the same letter could show up multiple times.
        # I also don't care if we check a position of the word that has already been guessed.
        # Notice, this also means you can guess the same correct letter multiple times.

# E. Print outcome of guess; adjust misses and past guesses
if guess_is_correct:
    print("Correct!\n\n")
else:
    total_misses += 1
    past_guesses.append(guess) # I opted for only tracking misses.
    print(f"Incorrect!\n\n")

# F. Check if game won/ game over.
if guessed_correctly == len(secret_word)*[True]: # Feels very ugly; but works.
    print("!!! You win !!!")
    break
if total_misses == total_misses_allowed:
    print("--- Game Over ---")
    break

# initialization - main body (iterations) - ending conditions
# Intended block!!!

```

```
_ _ _ _ _  
Already guessed: [].
```

Correct!

```
h _ _ _ _  
Already guessed: [].
```

Incorrect!

```
h _ _ _ _  
Already guessed: ['r'].
```

Correct!

```
h e _ _ _  
Already guessed: ['r'].
```

Correct!

```
h e l l _  
Already guessed: ['r'].
```

Correct!

```
h e l l _  
Already guessed: ['r'].
```

Correct!

!!! You win !!!

Comments on Hangman Game

- In general, this game is very complicated to code. I know it was very hard. But I also think it's worth trying to code something larger (even as early as week 5) as this is a skill you need to learn. Rest assured, if we had a mid-term next week (which we don't), I would never have asked you to solve a question like this on that mid-term exam!
- There are many ways to code this. Don't think my solution is the only one. (nor assume that I managed to code this in one go)
- In the future, we'll see how to write this much cleaner using functions and classes.